# Technical Requirements Document (TRD)

**Project:** Adaptive AI Exam Diagnostician ("The Panic Button")
**Target:** One-Day Batting Students / Crammers under Severe Constraints

## 1. Product Requirements Overview

The application functions as a **cognitive triage terminal** rather than a standard chat window or static note viewer. It catches a student's high-stress goals, strips away 80% of content noise, maps it straight to a domain-specific neurological framework, and establishes immediate active recall loops to guarantee rapid retention before exams.

**Core User Workflow:**

• **The Diagnostic Triage:** User logs in and provides core targets: Time remaining, Target grade, and Subject domain.

• **Resource Ingestion:** User uploads a raw file (PDF Question Bank/Syllabus) or provides a live web URL link.

• **Execution Delivery:** The app skips generic conversation and renders a hyper-optimized 3-Tab dashboard: Visual Cheat Sheet, Active Drill Quiz, and Oral/Viva Feedback Mirror.

## 2. Architectural & Data Flow Specifications

To optimize pipeline speed under 5 seconds, the data pipeline runs entirely inside massive LLM context windows (e.g., 128k+ tokens), bypassing traditional multi-step vector storage indexing workflows (RAG) for short/medium study sets.

**The Pipeline Layer Array:**

1. **Frontend Canvas:** A fast Next.js UI captures user criteria and resources.

2. **FastAPI Layer:** Manages session state and triggers parallel, concurrent data tasks.

3. **Extraction Cluster:** PyMuPDF parses files offline, while Crawl4AI Docker containers scrub dynamic URLs directly into raw markdown strings.

4. **Async Generation Engine:** Utilizes Python's asyncio to fire multiple concurrent LLM instructions to build cheat sheets, flowcharts, and recall items instantly.

## 3. Comprehensive Technology Stack

| Component | Selected Tech Stack | Selection Rationale |
|---|---|---|
| Frontend View | Next.js 15 (App Router) | Blazing fast SSR, dynamic client layouts, native state synchronization. |
| Styling / Graphs | Tailwind CSS + Mermaid.js | Tailwind designs fluid layouts. Mermaid.js renders structural charts directly from markdown text. |
| Backend Core | FastAPI (Python 3.11+) | Native asynchronous concurrency loops using asyncio. Reduces backend pipeline latency. |
| PDF Parsing | PyMuPDF (fitz) / Marker | Ultra-rapid processing of raw textbook layouts, structure, and formulas. |
| Web Scraping | Crawl4AI (Headless Docker) | Strips JS trackers, styles, and menus to yield lightweight markdown blocks. |
| JSON Enforcement | Instructor SDK | Rigidly maps Pydantic models to LLM outputs to guarantee zero runtime parsing errors. |
| AI Inference | OpenAI gpt-4o-mini / Groq | Sub-second token response times coupled with massive token context capacity. |
| Telemetry Base | Supabase (PostgreSQL) | Logs granular student failure telemetry vectors to automate reinforcement learning updates. |

# 4. Step-by-Step Data Engineering & Pipeline Specs

## 4.1 Rule-Based Mapping Matrix Configuration

The orchestration backend intercepts structural variables and pairs them with specific cognitive models instead of relying on generic open conversational responses:

```
# Cognitive Science Framework Routing Setup
COGNITIVE_STRATEGY_MATRIX = {
    "stem": {
        "framework": "Feynman Deconstruction & First Principles",
        "prompt_anchor": "Deconstruct every single technical formula into a plain, jargon
-free real-world analogy. Focus on logical constraints.",
        "ui_target": "flowchart"
    },
    "humanities": {
        "framework": "Dual Coding & Spatial Memory Palaces",
        "prompt_anchor": "Synthesize chronological text walls into visual matrix blocks,
grouping milestones by structural concept grids.",
        "ui_target": "grid_palace"
    }
}
```

## 4.2 High-Speed Async Orchestration Execution

The system bypasses synchronous delays by calling parallel micro-extraction steps simultaneously inside a single runtime thread block via the Instructor pattern:

```
import asyncio
from pydantic import BaseModel, Field
from typing import List
import instructor
import openai

class QuizItem(BaseModel):
    question: str = Field(description="High-yield exam question from context")
    options: List[str] = Field(description="Exactly 4 unique choice options")
    correct_answer: str = Field(description="The matching correct option string")
    explanation: str = Field(description="Short conceptual justification mapping to text"
)

class ExamTriagePayload(BaseModel):
    cheat_sheet_markdown: str
    concept_flowchart_mermaid: str
    diagnostic_quiz: List[QuizItem]

async def execute_parallel_cognitive_pipeline(scraped_text: str, strategy: dict) -> ExamT
riagePayload:
    client = instructor.from_openai(openai.AsyncOpenAI())

    response = await client.chat.completions.create(
        model="gpt-4o-mini",
```

```
        response_model=ExamTriagePayload,
        messages=[
            {"role": "system", "content": f"You are an elite academic architect using {st
rategy['framework']}."},
            {"role": "user", "content": f"Process this exam source data: \n{scraped_text[
:25000]}"}
        ]
    )
    return response
```

### 4.3 Continuous Memory Evaluation Schema

When student fail ratios on a specific concept vector pass a 40% threshold, a tracking index prompts the prompt matrix script to self-correct and switch to alternative visual rendering anchors.

```
-- SQL Telemetry Logging Ledger
CREATE TABLE student_telemetry_logs (
    interaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    concept_id VARCHAR(255) NOT NULL,
    cognitive_framework_used VARCHAR(50) NOT NULL,
    student_input_text TEXT NOT NULL,
    semantic_accuracy_score INT CHECK (semantic_accuracy_score BETWEEN 0 AND 100),
    user_satisfaction_bool BOOLEAN,
    logged_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_concept_failures
ON student_telemetry_logs (concept_id)
WHERE semantic_accuracy_score < 50;
```